TASK 06 – DESIGN THE CONTROL STRUCTURES

1. Aim

To design and evaluate decision-making and looping structures for calculating metro fares and processing daily journeys using C++.

2. Decision-Making Structures

Structure	**Application**
If–else	Checks whether the smart card has sufficient balance.
Switch	Selects the base fare according to the travel zone.
Conditional operator	Selects between regular and discounted fares.

3. Looping Structures

Loop	**Application**	**Suitability**
For loop	Processes a fixed number of journeys.	Suitable
Range-based for loop	Processes each journey in an array.	Best for the given program
While loop	Processes journeys until a condition becomes false.	Suitable for unknown journey counts.

4. Break and Continue

- **Break:** Terminates journey processing when an end-of-day or cancellation condition occurs.
- **Continue:** Skips an invalid journey and proceeds to the next journey.

5. Switch Default Case

If no switch case matches the selected zone, the default case handles the invalid zone and prevents an incorrect fare from being calculated.

6. Algorithmic Pseudocode

Algorithm: Metro Fare Computation

1. BEGIN
2. INPUT zone, peak status and card balance.
3. USE SWITCH to select the base fare according to the zone.
4. IF the zone is invalid, reject the journey.
5. IF peak hours apply, calculate the 20% surcharge using integer arithmetic.
6. CALCULATE the final fare.
7. IF remaining balance is at least ₹50:
    - Deduct the fare.
    - DISPLAY fare and remaining balance.
8. ELSE:
    - DISPLAY transaction denied.
9. REPEAT for every journey.
10. END

7. Alternative Concept and Complexity Analysis

Aspect	**Switch Statement**	**Array Lookup**
Fare selection	Uses zone cases.	Uses zone index.
Time complexity	O(1)	O(1)
Space complexity	O(1) additional space	O(n) for the fare array
Fourth zone	Requires another case.	Requires another array element.

Alternative Concept

Instead of using a switch statement, the base fare can be selected directly from an array using the zone index.

Alternative code:

```
constexpr long long FARE[] = {
    1000, 2000, 3000
};
long long baseFare = FARE[static_cast<int>(z)];
```

The surcharge calculation remains the same.

Complexity Analysis

Parameter	Switch Approach	Array Approach
Time complexity	O(1)	O(1)
Additional space	O(1)	O(1)
Storage	No lookup array required beyond the given base fare array	Uses a fare lookup array

Fourth zone	Add another switch case	Add another array element
Flexibility	Explicit zone handling	Easy indexed lookup

Evaluation: Both approaches provide constant-time fare selection. The switch approach is more readable for a small number of named zones, whereas array lookup is convenient when zones are represented by consecutive integer indices. Array indexing must be validated to prevent out-of-bounds access.

Result: The decision-making and looping structures were successfully designed for metro fare calculation and daily journey processing. The program demonstrates zone-based fare selection, peak-hour surcharge calculation, minimum balance validation, and journey processing using appropriate control structures.

Conclusion: Control structures provide efficient fare calculation and journey processing. A range-based for loop is preferred for the given journey array, while switch statements ensure clear zone-based fare selection. Integer paise calculations maintain monetary accuracy.